Two-Layer Agent Architecture

Orchestration and Specialization in Agentic Coding

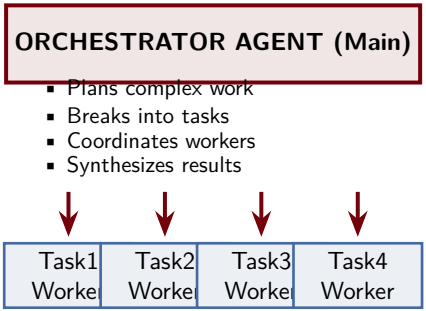
Teaching Agentic Coding Tools

University of South Carolina

January 23, 2026

Outline

Two-Layer Agent Architecture: Overview



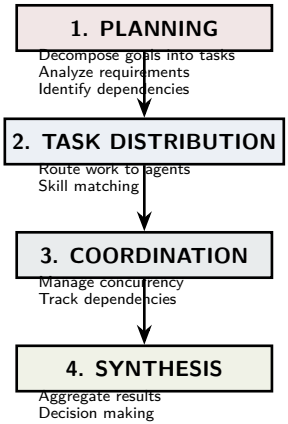
(Parallel Execution)

Key Concepts

- Modern agentic coding uses two layers
- **Orchestrator:** Strategic planning
- **Workers:** Focused execution
- Enables scaling and parallelization
- Mirrors human project management

Key Insight: Separation of concerns through orchestration-worker division

Orchestration: The Art of Coordinating Agents



Orchestrator Responsibilities

- Coordination layer managing multiple agents
- Decides **WHAT** work and **WHO** does it
- Does not execute work directly

Requires Understanding

- Problem decomposition
- Agent capabilities
- Dependency management
- Result integration

Roles in the Two-Layer System

MAIN AGENT (Orchestrator)

Responsibilities:

- Receives user request
- Analyzes scope
- Creates plan
- Spawns tasks/workers
- Waits for results
- Synthesizes output

Scope: System-wide view
Token Budget: Full context

WORKER AGENTS

Responsibilities:

- Receive task description
- Execute work
- Report results
- Handle errors
- Stay focused

Constraints: ∼200K tokens
Advantages: No context bloat

Key Distinction: Workers are ephemeral; Orchestrator is persistent

Benefits of Task Decomposition

Problem: Single Agent

- Context bloat (token limits)
- Loss of focus (too many concerns)
- Sequential execution (slow)
- Hard to isolate failures
- Cannot parallelize
- Reduced effectiveness

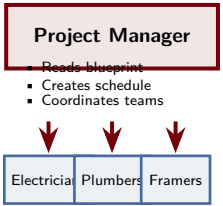
Solution: Orchestrator + Workers

- Clear token boundaries
- Task focus (one thing well)
- Parallel execution
- Isolated failure handling
- Easy to retry failed tasks
- Better result quality
- Scalability

Key Concept: Task splitting enables parallelization and focus

Two-Layer Pattern in Familiar Contexts

Construction Project

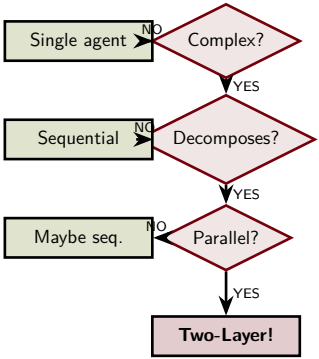


Magazine Production

Universal Pattern

- **Software:** Tech lead orchestrates backend, frontend, QA teams
- **AI Agents:** Orchestrator agent → specialist worker agents
- Pattern is timeless and proven at scale

When Should You Use Two-Layer Architecture?



Decision Tree

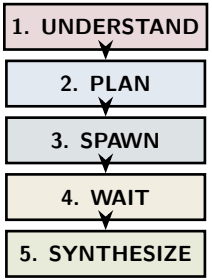
Use Two-Layer When:

- Task breaks into 3+ subtasks
- Subtasks are independent
- Total work $>$ 2–3 hours
- Subtasks need specialization
- Parallelization saves time

Avoid Two-Layer For:

- Simple, quick tasks ($<$ 1 hour)
- Heavy dependencies
- Constant decision-making
- Minimal context needs

The Orchestrator: Strategic Decision-Maker



Parse request, identify scope

Break into atomic tasks

Create workers with ins

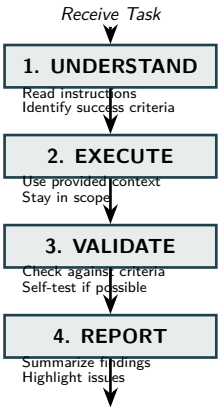
Collect results, handle timeouts

Combine into final answer

Key Principle:
Orchestrator =
Plan → Delegate → Integrate

Does not execute work,
only coordinates it.

Worker Agents: Focused Executors



Worker Lifecycle

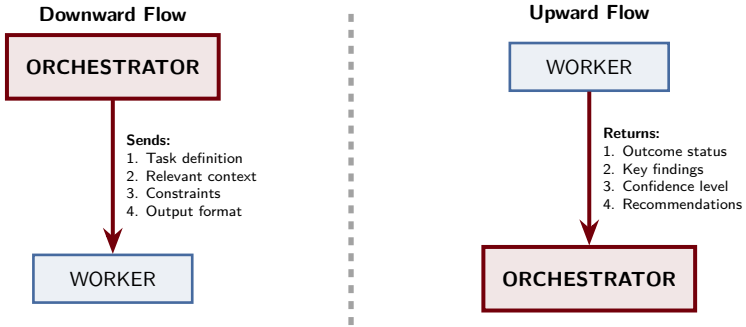
Worker Principles

- **Focused:** One task at a time
- **Disciplined:** Stay within scope
- **Efficient:** Use tokens wisely
- **Clear:** Structured reporting

Good Worker Behavior

- Reviews exactly assigned files
- Does not expand scope
- Notes out-of-scope issues
- Returns structured results

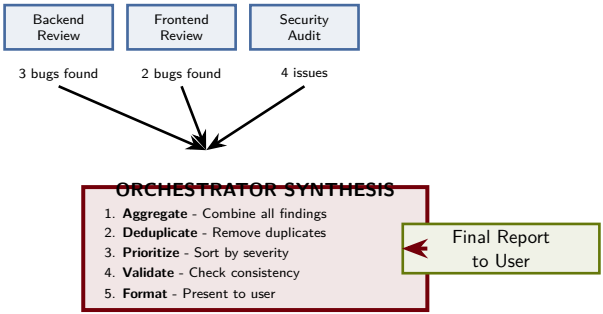
How Information Flows Between Layers



Communication Principle

Like API contracts: clear input specification and output schema for easy synthesis

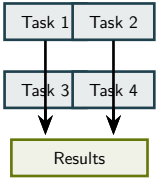
Combining Worker Results Into Final Output



Key: Synthesis = Integration + Analysis + Decision-Making

Handling Task Dependencies

Pattern 1: Parallel

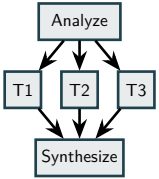


No dependencies
All run simultaneously

Pattern 2: Linear

Sequential deps
Must run in order

Pattern 3: Tree

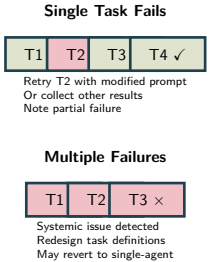


Mixed parallel
and sequential

Dependency Resolution

- Identify dependencies early in planning phase
- Run independent tasks in parallel; only serialize when necessary
- Use completed results to inform later tasks

Managing Failures in the Two-Layer System



Error Scenarios

Single Failure Retry or use partial results

Timeout Worker too slow; terminate and retry

Systemic All fail; indicates design problem

Recovery Strategies

- Retry with simpler prompt
- Add more context/guidance
- Break into smaller pieces
- Fallback to single-agent

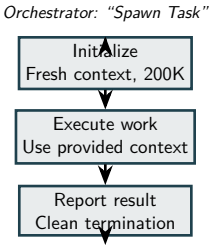
Two Ways to Delegate Work

Aspect	TASKS	SUBAGENTS
Lifetime	Ephemeral, one-off job	Persistent throughout sequence
Token Budget	∼200K per task, no carry-over	Fresh budget per interaction
State	Stateless, no memory	Maintains state, remembers
Init Cost	Quick, minimal overhead	Slower, more setup
Ideal For	Isolated, parallelizable work	Iterative, multi-turn work
Concurrency	Up to 10 simultaneous	Typically 1–3 sequential

Key Distinction

Tasks = Stateless one-offs for parallel execution

Tasks: Lightweight, Stateless Execution



Ideal Task Characteristics

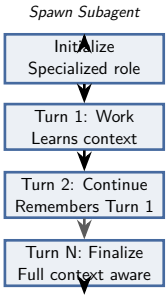
- Self-contained work
- Clear input $\rightarrow$ output
- No previous task dependency
- Can be parallelized
- Execution $<$ 5 minutes
- Result is final (no back-and-forth)

Best For

Analyzing files, running tests, generating docs, code reviews, data processing

Advantage: Fresh context means no bloat; optimized for parallelization

Subagents: Persistent, Stateful Partners



Ideal Subagent Work

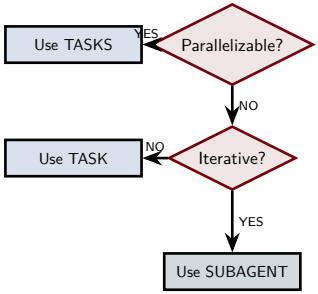
- Iterative problem solving
- Multi-turn interactions
- Specialist roles
- Benefits from context growth
- Related decisions over time

Best For

Architecture design, writing with feedback, complex debugging, feature implementation cycles

Advantage: Remembers prior interactions, builds sophisticated understanding

Decision Tree: Tasks or Subagents?



Quick Reference

Task	Subagent
Code review	Code architect
Test file X	Integrate & test
Security scan	Design protocol
Analyze data	Data strategy
Generate docs	Doc specialist

Rule of Thumb:
Parallelizable = Tasks
Iterative = Subagents

The Task Tool: Spawning Worker Agents

What It Is

- Built-in command for parallel agents
- Terminal-based, integrated
- Handles context isolation
- Manages constraints and execution

Key Features

- Parallel execution (up to 10 concurrent)
- Automatic context isolation
- Result collection and buffering
- Timeout management
- Structured output format

Basic Syntax

```
task: "instruction" [options]
```

Example

```
task: "Review auth module"
  -context="src/auth.py"
  -timeout=300
  -name="auth-review"
```

Key Concept:
Task tool = Built-in parallelization mechanism

Task Tool Parameters and Options

Complete Syntax

```
task: "Primary instruction"
  -name="task-identifier"
  -context="file or glob"
  -timeout=300
  -output-format="json"
  -priority="normal"
```

Full Example

```
task: "Find security vulns"
  -name="security-audit"
  -context="src/auth/*.py"
  -timeout=300
  -output-format="json"
  -priority="high"
```

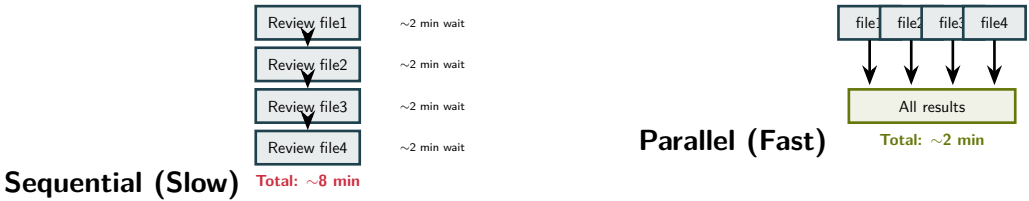
Parameter Details

Instruction	What to do (required)
Name	Identifier for tracking
Context	Files to provide
Timeout	Max seconds (120–600)
Output	

Best Practice

Provide only relevant context. If task reviews `auth.py`, provide just that file.

Running Tasks in Parallel



Parallelization Efficiency

- **Constraint:** Maximum 10 concurrent tasks
- **Speedup:** N similar tasks take $\sim 1/N$ time
- **Strategy:** Spawn all at once, wait for batch completion

Token Budget and Context Isolation

Task Budget:
Instruction: ~0.1K
Context: ~50K
Working: ~149K
Output: ~0.9K

Good Context

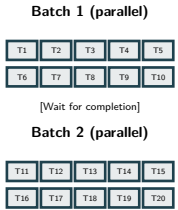
`-context="src/auth.py"` [5K]
Leaves plenty for thinking

Bad Context

`-context="src/**/*.py"` [500K]
Exceeds budget, will fail

Concurrency Limits and Batching

Maximum 10 Concurrent



Batching Strategy

50 functions to review:

- Batch 1: Func 1–10 (2 min)
- Batch 2: Func 11–20 (2 min)
- Batch 3: Func 21–30 (2 min)
- Batch 4: Func 31–40 (2 min)
- Batch 5: Func 41–50 (2 min)

Total: 10 min vs 100 min sequential

Constraint

Hard limit of 10 concurrent tasks. If >10 needed, batch sequentially.

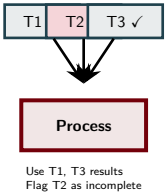
Handling Task Results

Result Structure

```
{
  "task_id":  "t1",
  "status":  "success",
  "output":  "result",
  "exec_time":  45,
  "tokens_used":  12500
}
```

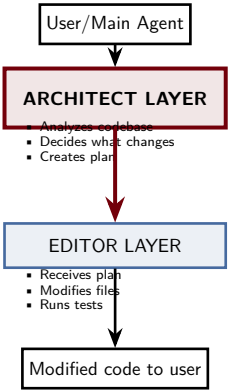
Status Values

success	Process normally
timeout	Ran too long
error	Something failed



Key: Collect all results, handle failures gracefully

Two-Layer Pattern in Aider



Aider Roles

Architect Analyzes, plans, decides approach

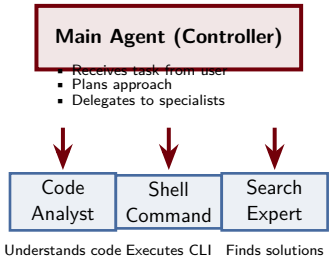
Editor Implements specified changes

Analogy to Claude Code

- Architect = Orchestrator
- Editor = Task Worker
- Both use Claude LLM

Key: Separation of thinking vs doing

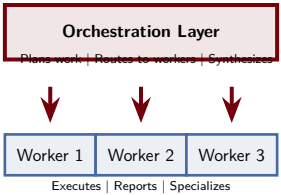
Delegation Pattern in OpenHands



Specialist Pattern

- Each agent handles specific domain
- Controller routes tasks to appropriate specialists
- New specialists can be added without changing core

Common Two-Layer Patterns Across AI Tools



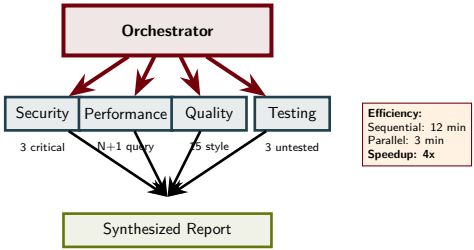
Why This Pattern Works

- Separation of concerns
- Specialists > generalists
- Easy to scale (add workers)
- Failure isolation
- Parallelization

Tool Comparison

Tool	Pattern
Claude Code	Main → Tasks
Aider	Architect → Editor
OpenHands	Controller → Agents
Copilot	Pipeline stages

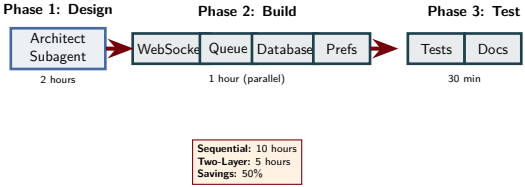
Example: Multi-Specialist Code Review



Pattern Benefits

- Each specialist focuses on one domain
- 4 experts work simultaneously on same code
- Combined findings into prioritized report

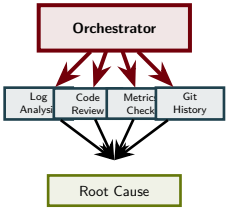
Example: Building a Complex Feature



Key Insight

Large features split into parallelizable components, then recombined

Example: Distributed Bug Investigation



Findings Converge

- Logs: Memory allocation failure
- Code: No streaming, full load
- Metrics: Memory 100% at crash
- Git: Removed streaming code

Root Cause

Optimization removed streaming + new dependency = OOM on large payloads

Time: 10 min vs 2 hours (90% saved)

Design Principles for Two-Layer Systems

Orchestrator Best Practices

1. **Plan thoroughly**
   Identify parallelizable work early
2. **Provide context efficiently**
   Only what workers need
3. **Write clear instructions**
   Specific, measurable criteria
4. **Handle failures gracefully**
   Expect some workers to fail

Worker Best Practices

1. **Understand scope**
   Stay focused on assigned task
2. **Use provided context**
   Do not expand beyond it
3. **Validate output**
   Check against criteria
4. **Report clearly**
   Structured, concise findings

Core Principles: Specialization | Independence | Isolation | Clarity | Efficiency

Common Pitfalls and How to Avoid Them

Over-Parallelization

Problem: Spawn 100 tasks for simple job
Fix: Use only for 3+ substantial tasks

Poor Task Definition

Problem: "Do something smart"
Fix: Specific instructions with criteria

Context Bloat

Problem: Entire codebase to each worker
Fix: Only relevant files within 200K

Unhandled Dependencies

Problem: Parallel tasks that depend on each other
Fix: Identify deps early, serialize when needed

Ignoring Failures

Problem: No fallback when workers fail
Fix: Use partial results, retry, or redesign

Too Many Tasks

Problem: Spawn 50 tasks at once
Fix: Batch into groups of ≤ 10

Measuring Success: Key Metrics

Metric	Description	Target
Time Efficiency	Sequential / Parallel time	3–8x speedup
Quality	Result improvement vs single agent	20–40% better
Cost	Tokens used per work unit	$\leq$ sequential
Reliability	Worker success rate	$>$90%
Scalability	Time vs problem size	Sublinear

Decision Framework

- Speedup $>$ 2x? Continue with two-layer
- Speedup $<$ 1.5x? Reconsider design
- Failure $>$ 10%? Task definitions need work

Two-Layer Agent Architecture: Summary

Core Concept

Orchestrator-Worker Pattern

- Orchestrator: Plans, delegates, decides
- Workers: Execute, report, specialize
- Together: Solve complex problems fast

When to Use

- Complex, decomposable work
- 3+ independent tasks
- Total work $>$ 2–3 hours
- Parallelization beneficial

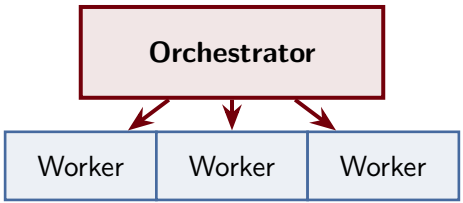
Key Benefits

- Parallelization (3–8x speedup)
- Specialization (20–40% quality)
- Scalability (easy to add workers)
- Resilience (isolated failures)

Claude Code Implementation

- Main agent = Orchestrator
- `task: "..."` = Worker spawn
- Up to 10 concurrent
- $\sim$200K tokens each

Questions?



Two-Layer Agent Architecture

Teaching Agentic Coding Tools